

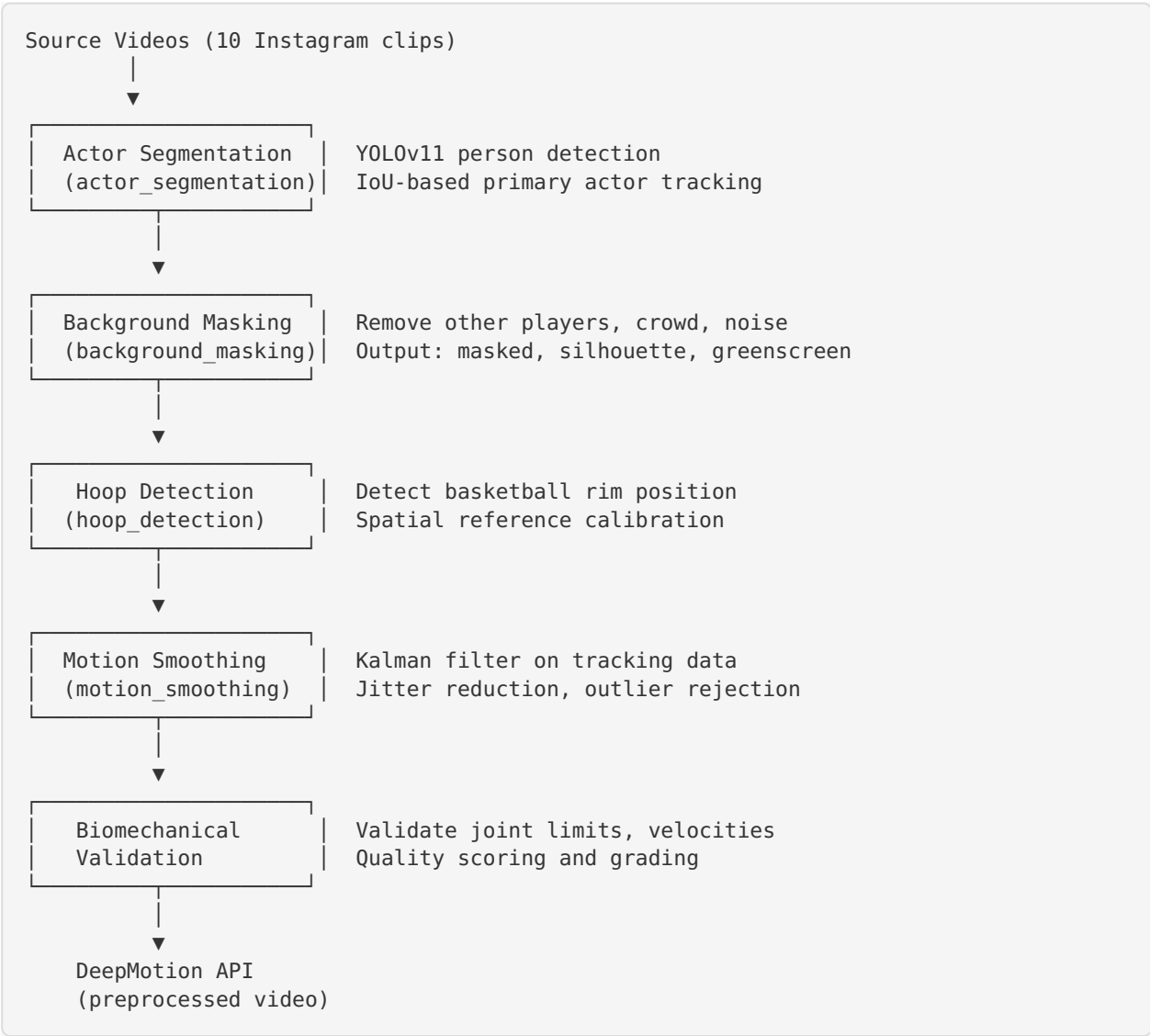
CV Preprocessing Pipeline — Biomechanical Isolation

Overview

The CV Preprocessing pipeline isolates the primary actor (@elijahbonds) from complex basketball game footage before sending videos to DeepMotion for motion capture. This prevents **tracking hallucinations** caused by:

- Multiple players in frame
- Crowd/spectator movement
- Camera shake and environmental noise
- Scoreboard overlays and graphics

Architecture



Modules

1. Actor Segmentation (actor_segmentation.py)

Purpose: Detect and track the primary actor across all video frames.

Methods:

- **YOLOv11** (primary): Ultra-fast person detection with high accuracy
- **SAM** (optional): Segment Anything Model for pixel-precise masks
- **OpenCV MOG2** (fallback): Background subtraction when no GPU available

Primary Actor Selection:

1. First frame: Largest bounding box (assumes primary actor is most prominent)
2. Subsequent frames: IoU-based tracking with center-distance fallback
3. Gap handling: Up to 10 frames of tracking loss before re-initialization

```
# Single video
python actor_segmentation.py --input video.mp4 --output masks/

# With SAM (GPU required)
python actor_segmentation.py --input video.mp4 --output masks/ --use-sam
```

2. Background Masking (background_masking.py)

Purpose: Create clean videos with only the primary actor visible.

Output Modes:

Mode	Description	Use Case
masked	Actor pixels + black background	DeepMotion input
silhouette	White actor on black	Pose visualization
greenscreen	Actor on green background	Compositing
alpha	BGRA with transparency	Advanced editing

Features:

- Edge feathering for smooth actor boundaries
- Temporal mask consistency
- Configurable background color

3. Hoop Detection (hoop_detection.py)

Purpose: Detect basketball hoop position as a spatial reference anchor.

Detection Strategies:

1. **Color-based:** HSV filtering for orange rim
2. **Hough circles:** Circular shape detection
3. **Contour-based:** Rectangular backboard detection
4. **Temporal averaging:** Stabilize detection across frames

Spatial Calibration:

- Standard rim diameter = 18 inches → pixels-per-inch conversion
- Standard rim height = 10 feet → height reference
- Enables validation of actor reach/jump heights

4. Motion Smoothing (`motion_smoothing.py`)

Purpose: Reduce tracking jitter and noise in motion data.

Filters Available:

Method	Best For	Latency
<code>kalman</code>	Real-time tracking, predictive	Low
<code>savgol</code>	Post-processing, preserves peaks	None
<code>ema</code>	Simple temporal smoothing	Low
<code>combined</code>	Kalman + Savitzky-Golay	None

5. Biomechanical Validator (`biomechanical_validator.py`)

Purpose: Verify motion data is physically plausible.

Checks:

- ☒ Joint angle limits (20 joint types)
- ☒ Velocity plausibility (no teleportation)
- ☒ Tracking continuity (gap detection)
- ☒ Bounding box consistency (identity swap detection)
- ☒ Spatial reference consistency
- ☒ Smoothing effectiveness
- ☒ Mask coverage quality

Quality Grading:

Score	Grade	DeepMotion Ready
90+	A	☒ Yes
75-89	B	☒ Yes
60-74	C	☒ Yes (with warnings)
40-59	D	☒ Review needed
<40	F	☒ Re-process

Usage

Process All Videos

```
cd /path/to/rork-final-evolution-lab
python scripts/cv_preprocessing/preprocessing_pipeline.py --all
```

Process Single Video

```
python scripts/cv_preprocessing/preprocessing_pipeline.py \
  --video 01_dunk_session_multiple_dunks_reverse_alleyoop.mp4
```

Full Pipeline with Validation

```
# Run as part of master pipeline
bash scripts/ue5_setup/fel_complete_pipeline.sh --cv-only
```

Custom Configuration

```
python scripts/cv_preprocessing/preprocessing_pipeline.py --all \
  --method combined \
  --confidence 0.3 \
  --use-sam \
  --hoop-sample-rate 3
```

Output Structure

```
GeneratedAssets/CVPreprocessed/
├── metadata/
│   ├── 01_dunk_session_cv_meta.json
│   ├── 02_running_jump_cv_meta.json
│   └── ...
├── 01_dunk_session_multiple_dunks_reverse_alleyoop/
│   ├── segmentation/
│   │   ├── masks/                # Per-frame binary masks
│   │   └── *_segmentation_meta.json
│   ├── masked/
│   │   ├── *_masked.mp4          # → Send to DeepMotion
│   │   └── *_silhouette.mp4
│   ├── hoop/
│   │   ├── annotated/           # Visual hoop detection
│   │   └── *_hoop_meta.json
│   ├── smoothed/
│   │   └── *_smoothed.json
│   └── ...
```

Dependencies

```
pip install ultralytics opencv-python-headless scipy numpy
# Optional (GPU):
pip install segment-anything torch torchvision
```

Integration with DeepMotion

The pipeline automatically recommends the best input for DeepMotion:

1. **Preprocessed video** (preferred): Masked video with isolated actor
2. **Original video** (fallback): When preprocessing detection rate > 80%
3. **Manual review** (rare): When quality score < 60

The `process_elijahbonds_animations.py` script checks for preprocessed videos:

```
cv_output = f"GeneratedAssets/CVPreprocessed/{video_name}/masked/{video_name}
_masked.mp4"
if os.path.exists(cv_output):
    video_to_process = cv_output # Use preprocessed
else:
    video_to_process = original_path # Fallback to raw
```

Troubleshooting

See [TROUBLESHOOTING.md](#) (TROUBLESHOOTING.md#cv-preprocessing) for common issues.